

CO27-0852/2024

WRITTEN QUESTIONS

Question 1: Design Decisions

(i) What is your admission number, and how did it determine your dataset (number of gigs, categories, and currency)?

Answer:

My admission number is C027-01-0852/2024. Based on the assignment instructions, it determined that my dataset would contain 7 gigs, use the categories Development, Design, and Writing, and use KES as the currency.

(ii) Why did you choose to include the status field in your gig data model? What are the possible statuses and what do they represent?

Answer:

I included the status field to show the progress of each gig. The possible statuses are Open, In Progress, and Closed, which indicate whether a gig is available, currently being worked on, or already completed.

Question 2: Validation

(i) What validation rules did you apply to the budget field in your Pydantic model? Why is this validation important?

Answer:

I validated the budget field by requiring it to be greater than zero. This is important because it prevents users from entering invalid values such as zero or negative budgets.

(ii) How did you ensure that the category field only accepts values from your assigned category list?

Answer:

I created a list of allowed categories and checked whether the submitted category was in that list. If the category is not valid, the API returns a 400 Bad Request error.

Question 3: Error Handling

(i) What status code does your API return when a user tries to retrieve a gig that does not exist? Explain why this is the appropriate status code.

Answer:

The API returns a 404 Not Found status code when a gig does not exist. This is the correct status code because the requested resource cannot be found.

(ii) What status code does your API return when a user tries to create a gig with invalid data? Explain why.

Answer:

The API returns a 400 Bad Request status code when invalid data is submitted, such as an invalid category. This tells the user that the request contains incorrect information that needs to be corrected.

Question 4: Search and Filtering

(i) Describe how your /gigs/search endpoint works. How does it handle searches where the query string does not match any gig?

Answer:

The /gigs/search endpoint searches for gigs by comparing the search query with each gig's title. If no matching gig is found, it simply returns an empty list.

(ii) Describe how your /gigs endpoint with filtering works. What are the optional query parameters, and how do they work?

Answer:

The /gigs endpoint allows users to filter gigs using the optional parameters category, min_budget, and max_budget. These parameters return only the gigs that match the selected category or fall within the specified budget range.

Question 5: Code Walkthrough

(i) Write a brief explanation of the PUT /gigs/{gig_id} endpoint. What does it do, and how does it handle a request to update a gig that does not exist?

Answer:

The PUT /gigs/{gig_id} endpoint updates the budget or status of an existing gig. If the specified gig ID is not found, the API returns a 404 Not Found response.

(ii) Write a brief explanation of the POST /gigs endpoint. What does it do, and how does it ensure that the new gig is created correctly?

Answer:

The POST /gigs endpoint creates a new gig using the information provided by the user. Before adding the gig, it checks that the category is valid and confirms that another gig with the same title and client name does not already exist.

